SYSTEM ARCHITECTURE & ENGINEERING DESIGN SPEC

HIGH-CONCURRENCY TICKET BOOKING PLATFORM

Topic: Real-time Seat Hold, Redis TTL Expiration, Concurrency Locking & Waitlist Automation | **Target Word Count:** ~800 words

1. Core Architecture & Problem Statement

High-demand ticketing systems face severe operational bottlenecks: extreme traffic spikes during initial sale openings, race conditions from simultaneous seat bookings, and wasted inventory caused by abandoned checkouts. This architecture resolves these issues by pairing an event-driven Express/Node.js backend with Redis in-memory concurrency controls and PostgreSQL for persistent state management.

2. Dynamic Seat Hold & Redis TTL Mechanism

To prevent overbooking while allowing customers time to enter payment details, seats are placed in a temporary `HELD` state upon selection.

- **Hold Request Execution:** When a client issues a hold request, the backend creates a Redis key with an explicit Time-To-Live (TTL):
  `SET hold:event:{eventId}:seat:{seatId} {userId} EX 600`
  This key acts as the authoritative state for active holds during the 10-minute reservation window.
- **Automated Expiration via Pub/Sub:** Redis is configured to emit keyspace notifications (`notify-keyspace-events Ex`). A dedicated background worker process subscribes to the `__keyevent@0__:expired` channel. When a user abandons their session and the 600-second TTL expires:
  1. Redis fires an eviction event containing the expired key name.
  2. The worker parses the `eventId` and `seatId` from the channel payload.
  3. A database command updates the corresponding `EventSeat` record from `HELD` back to `AVAILABLE` .

3. Concurrency Prevention & Race Condition Safeguards

Preventing double-allocation of identical seats under simultaneous multi-user requests requires two synchronization layers working in sequence:

Layer A: Redis Distributed Lock (In-Memory Guard)

Before entering database operations, the API attempts to acquire an explicit distributed lock per requested seat:

```
SET lock:event:{eventId}:seat:{seatId} {requestId} NX EX 10
```

- `NX` : Ensures the lock key is created ONLY if it does not already exist.
- `EX 10` : Places a short 10-second safety fallback timeout on the lock itself to avoid deadlocks during server faults.

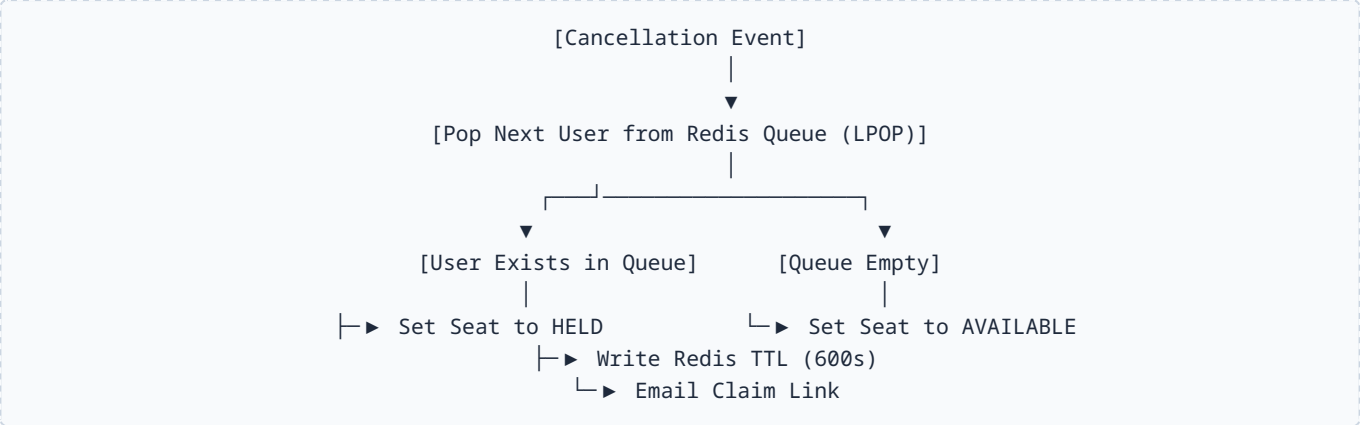
If any seat in a requested batch fails to acquire its Redis lock, the system immediately rejects the transaction with an HTTP `409 Conflict` status, releasing all previously claimed locks in that request batch.

Layer B: Database Transactional Isolation (Persistent Guard)

Once Redis locks are secured, execution moves into an isolated SQL transaction (`prisma.$transaction`). The query fetches seat records with strict conditional criteria (`WHERE status = 'AVAILABLE'`). If all target seat IDs remain unbooked, their status shifts atomically to `HELD` .

4. Waitlist Auto-Assignment Queue & Time-Limited Offers

When all seats in a given category reach `BOOKED` or `HELD` status, direct reservation closes and waitlist registration opens.



1. Waitlist Ingestion

Waitlist entries are queued per seat category using Redis FIFO Lists:
`RPUSH waitlist:event:{eventId}:category:{categoryId} {userId}`

2. Auto-Reallocation Engine on Cancellation

When a confirmed booking is cancelled:

1. The transaction handler executes an `LPOP` command against the event-category waitlist key.
2. **If a waitlisted user is retrieved:**
   - The system bypasses public release and assigns the seat directly into a targeted `HELD` state for that user.
   - A fresh 10-minute TTL key is written to Redis (`hold:event:{eventId}:seat:{seatId}`).
   - An asynchronous job dispatches an email notification containing a signed, time-limited checkout URL (`/checkout?token=...`).
3. **If the user claims the offer:** The payment completes and transitions the seat to `BOOKED`.
4. **If the user ignores the offer:** The 10-minute TTL expires, the worker catches the eviction event, and automatically triggers the reallocation loop again for the *next* candidate in the queue.

> **Summary Impact:** This architecture guarantees zero double-bookings under extreme concurrency, ensures automatic inventory recovery upon payment abandonment, and maintains maximum capacity utilization through waitlist automation.